CS6650 Midterm Mystery

Sai Karthikeyan Sura
March 8th, 2026

Overview

Hummingbird is a media management REST API (Node.js, Express, S3, DynamoDB, SNS) deployed to ECS Fargate behind an ALB via Terraform. Using Claude Code (v2.1.71, Opus 4.6 via AWS Bedrock) in CloudShell, I investigated, fixed, redeployed, and verified each bug against the live system.

Ticket #1: Server started on the wrong port

File: `server.js`, line 35.

Explanation: The port is read from `process.env.APP_PORT` with no fallback. If unset, `port` is `undefined`. Node.js treats `app.listen(undefined)` as port 0, so the OS assigns a random ephemeral port. The server starts silently on the wrong port, breaking ALB health checks.

```
// BEFORE:
const port = process.env.APP_PORT;

// AFTER:
const port = process.env.APP_PORT || 9000;
```

Ticket #2: Width is missing from metadata

File: `clients/dynamodb.js`, line 84 (in `getMedia`).

Explanation: `createMedia` stores `width` as a DynamoDB Number attribute, but `getMedia` omits it from the return object. The field exists in the database but is never read back.

```
// BEFORE (getMedia return):
return {
  mediaId,
  size: Number(Item.size.N),
  name: Item.name.S,
  mimetype: Item.mimetype.S,
  status: Item.status.S,
};

// AFTER:
return {
  mediaId,
  size: Number(Item.size.N),
  name: Item.name.S,
  mimetype: Item.mimetype.S,
  status: Item.status.S,
  width: Number(Item.width.N),
};
```

Ticket #3: Redirect URL is broken

File: `controllers/media.js`, line 111 (in `downloadController`).

Explanation: The Location header uses `req.hostname`, which returns only the hostname without a protocol scheme. A valid Location header must be an absolute URI starting with `http://`. The fix prepends the scheme and uses `req.get('host')` to include the port.

```
// BEFORE:
res.set('Location',
  `${req.hostname}/v1/media/${mediaId}/status`);

// AFTER:
res.set('Location',
  `http://${req.get('host')}/v1/media/${mediaId}/status`);
```

Ticket #4: Download never redirects when COMPLETE

File: `controllers/media.js`, line 108 (in `downloadController`).

Explanation: The condition checks `media.status !== MEDIA_STATUS.PROCESSING`, which is inverted. The status lifecycle is PENDING, then PROCESSING, then COMPLETE. With this condition, COMPLETE (which is not PROCESSING) returns 202 instead of the 302 redirect. It should check for COMPLETE.

```
// BEFORE:
if (media.status !== MEDIA_STATUS.PROCESSING) {

// AFTER:
if (media.status !== MEDIA_STATUS.COMPLETE) {
```

Bonus: Status never changes, no errors

File: `clients/dynamodb.js`, line 155 (in `setMediaStatus`).

Investigation: Comparing the Sort Key (SK) across all three DynamoDB functions:

```
createMedia:    SK: { S: 'METADATA' }  // uppercase
getMedia:       SK: { S: 'METADATA' }  // uppercase
setMediaStatus: SK: { S: 'metadata' }  // lowercase
```

DynamoDB keys are case-sensitive. `setMediaStatus` targets a different item than what `createMedia` wrote. When `UpdateItem` is called on a non-existent key with no `ConditionExpression`, DynamoDB performs an upsert, silently creating a phantom orphan item with SK `metadata`. The real record at SK `METADATA` is never updated. This is why there are zero errors in the logs.

CloudWatch logs confirmed: `getMedia` reads SK `METADATA` (finds the item), then both `setMediaStatus` calls write to SK `metadata` (wrong item). Status stays PENDING.

```
// BEFORE:
SK: { S: 'metadata' },

// AFTER:
SK: { S: 'METADATA' },
```

Verification

Step	Result	Validates
`GET /health`	200 OK	API is up
`POST /upload?width=800`	202	Upload works
`GET /media/:id`	200, width: 800	Ticket #2
`PUT /media/:id/resize`	200, COMPLETE	Bonus
`GET /media/:id/status`	200, COMPLETE	Bonus
`GET /media/:id/download`	302, valid URL	Tickets #3, #4

Complete Git Diff

```
diff --git a/clients/dynamodb.js b/clients/dynamodb.js
index 700494c..4291536 100644
--- a/clients/dynamodb.js
+++ b/clients/dynamodb.js
@@ -81,6 +81,7 @@
       name: Item.name.S,
       mimetype: Item.mimetype.S,
       status: Item.status.S,
+      width: Number(Item.width.N),
     };
   } catch (error) {
     logger.error(error);
@@ -151,7 +152,7 @@
     TableName,
     Key: {
       PK: { S: `MEDIA#${mediaId}` },
-      SK: { S: 'metadata' },
+      SK: { S: 'METADATA' },
     },
     UpdateExpression: 'SET #status = :newStatus',

diff --git a/controllers/media.js b/controllers/media.js
index 6bb1c50..8f3b61f 100644
--- a/controllers/media.js
+++ b/controllers/media.js
@@ -105,10 +105,10 @@
       return;
     }
-    if (media.status !== MEDIA_STATUS.PROCESSING) {
+    if (media.status !== MEDIA_STATUS.COMPLETE) {
       const SIXTY_SECONDS = 60;
       res.set('Retry-After', SIXTY_SECONDS);
-      res.set('Location',
-        `${req.hostname}/v1/media/${mediaId}/status`);
+      res.set('Location',
+        `http://${req.get('host')}/v1/media/${mediaId}/status`);

diff --git a/server.js b/server.js
index 454790a..46ce716 100644
--- a/server.js
+++ b/server.js
@@ -32,7 +32,7 @@
-const port = process.env.APP_PORT;
+const port = process.env.APP_PORT || 9000;
```